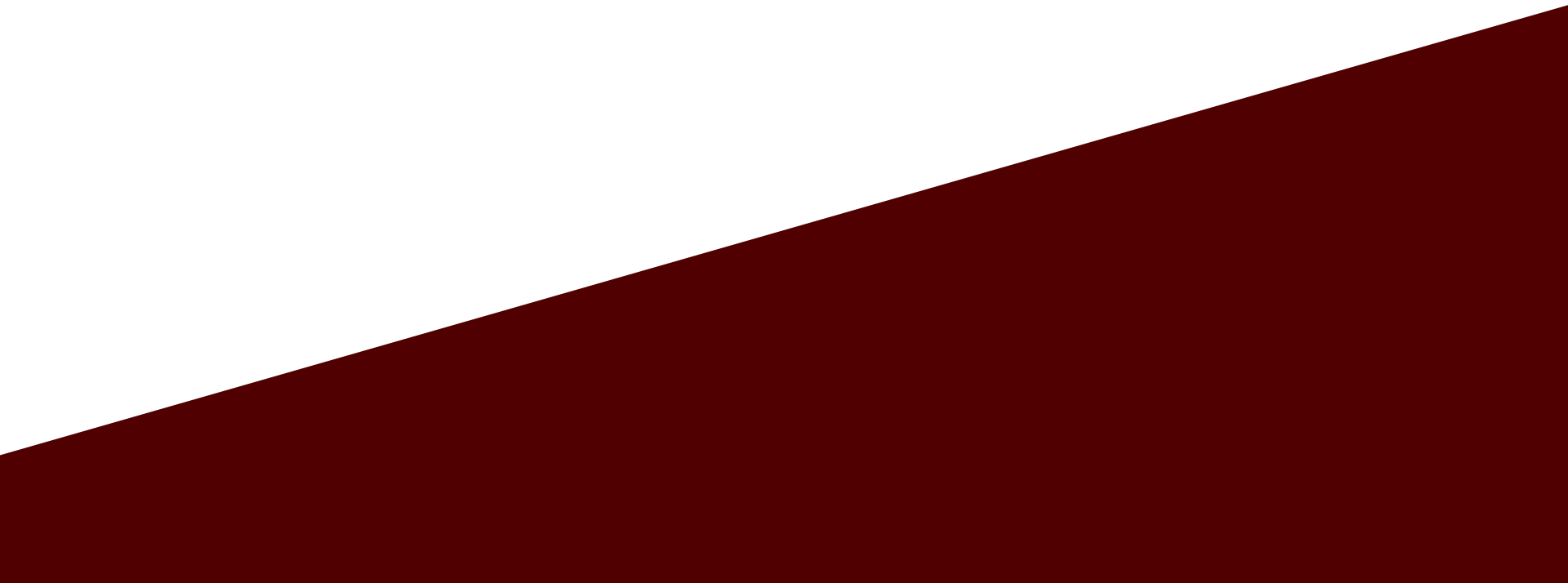


Lab 2 Introduction

CSCE 313: Introduction to Computer Systems



Objective

- Important concepts
- Lab 2 overview
 - Client ← what you will be modifying
 - Servers
 - Finance
 - File
 - Logging
 - RequestChannel class
 - Common class
 - Request
 - Response
 - RequestType
- Get familiar with processes and system calls like `fork()` + `exec(...)`

Important concepts

Program vs Process

- Program: **Executable** code
 - It sits lifeless on the hard drive as a “bag of bits”
- Process: An **instance** of a *program in execution*
 - For example: you could have many *instances* of a single program running on your system
- Each program runs in the **context** of some process
 - Context is described by process ID, processor state, and address space assigned to the process

Process Context

- Two important parts of process context for this lab
 - Process ID (and parent process ID) to uniquely identify current processes
 - Address Space is unique to each process
 - Stack: local variables, function params, etc.
 - Heap: dynamically allocated with malloc and new
 - Data: global/static variables, allocated when program starts
 - Code: instructions

Client Server Model

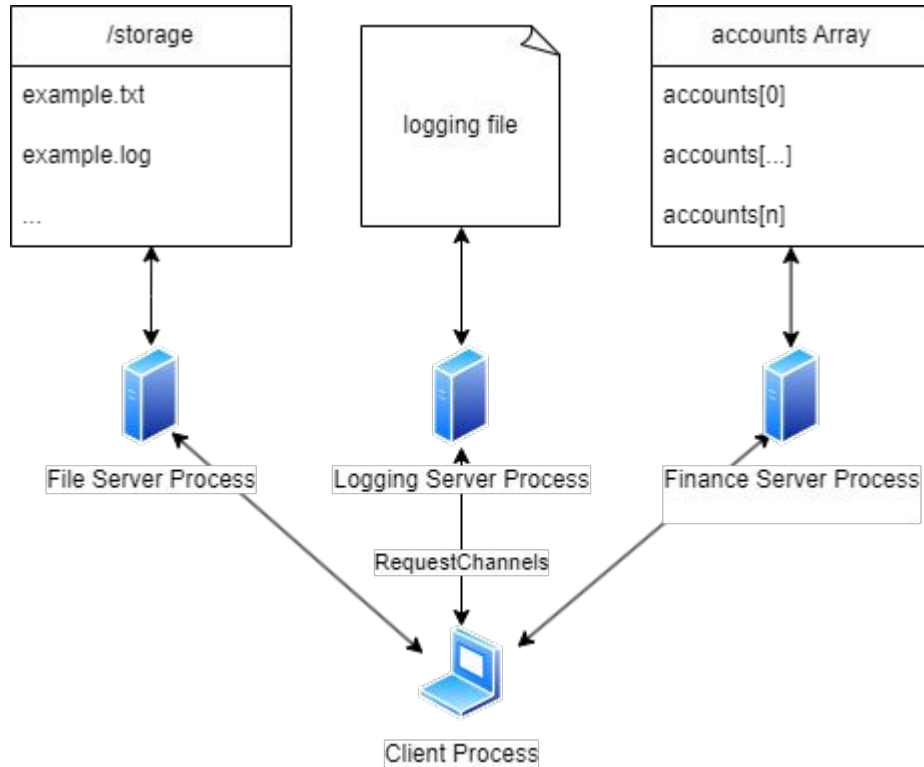
- A distributed architecture where clients (users or devices) request resources or services from a central server.
- Roles:
 - Client: Sends requests for data or services (e.g., browser, app).
 - Server: Processes requests and responds with the requested data or performs the service.
- Example: Web browsing (browser = client, web server = server)
- Advantages:
 - Centralized management of resources.
 - Easy to update/scale servers independently of clients.
- Challenges:
 - Single point of failure (server issues can disrupt service).
 - Requires robust server-side infrastructure to handle multiple clients.

Lab 2: *An overview*

Lab 2 Overview

- Multi-Server-Single Client architecture. We run the client program, which spawns three servers.
- Each server has a different responsibility
 - File: handles uploads and downloads to storage directory
 - Finance: handles users' accounts
 - Logging: logs user actions in a file
- Your task is to implement the client which communicates with a server to perform user specified tasks
- The server client and servers follow a communication protocol to:
 - Send requests to the server asking it for particular types of data
 - Retrieve and read the server's response

Lab 2 Architecture Diagram



Common class

- The Request class defines a RequestType, user_id, amount, filename, and data (string).
 - Not all of these fields are necessary for all request types. For example, a deposit request does not need a filename or data
 - Every Request must specify a RequestType, but all other fields are optional
- The Response class defines success (boolean), balance (double), data (string), and message (string)
 - Success determines whether the action requested was successful
 - Balance specifies the balance if the request was to the finance server
 - Data is the data associated with the response if the request needs it
 - Message is a short explanation of the completed action
- The enum RequestType defines types of Requests, such as QUIT, DEPOSIT, and LOGIN.

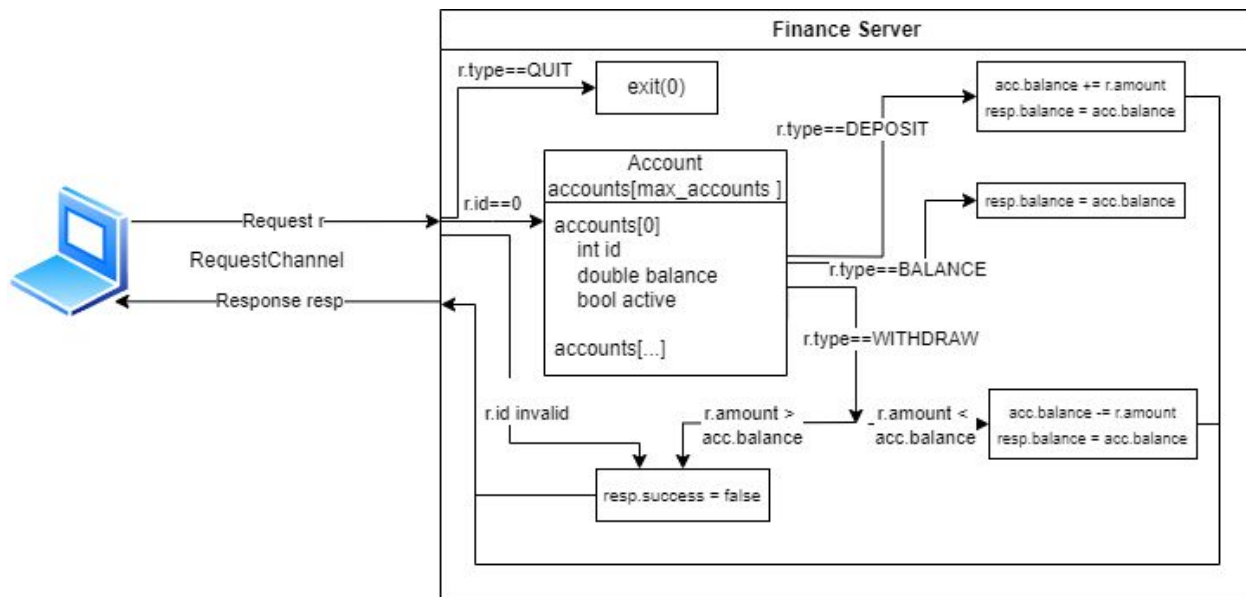
Servers

- All servers are fully implemented. You are not required to change the servers.
- All servers can handle a QUIT request, which will kill them
- You need to know how to format Requests and what information is important in the Response.
 - Each server handles some RequestTypes. Based on the type, they will use relevant fields in the Response in their logic. These fields must be filled out.
 - Similarly, Responses may have none, some, or all fields filled out when returning from the server. Examine the server code to understand what information you will get back.
- Beyond that, you do not need to know the details of how each server is implemented.

Finance Server

- The finance server has access to an Account array which tracks <n> users' accounts
- The number of accounts depends on the value passed when executing the server (0-max inclusive)
 - `./finance -m <max_account_num>`
 - Will not accept requests from users outside this range. Returns a false response
- The server can handle 3 request types (defined in the ***“common”*** class):
 - **DEPOSIT** - add an amount to the balance
 - **WITHDRAW** - attempt to subtract an amount from the balance. If larger than the balance, request fails
 - **BALANCE** - request the balance of the account
- If some other request type is sent to it, the server responds with false
- You are not required to change the server program

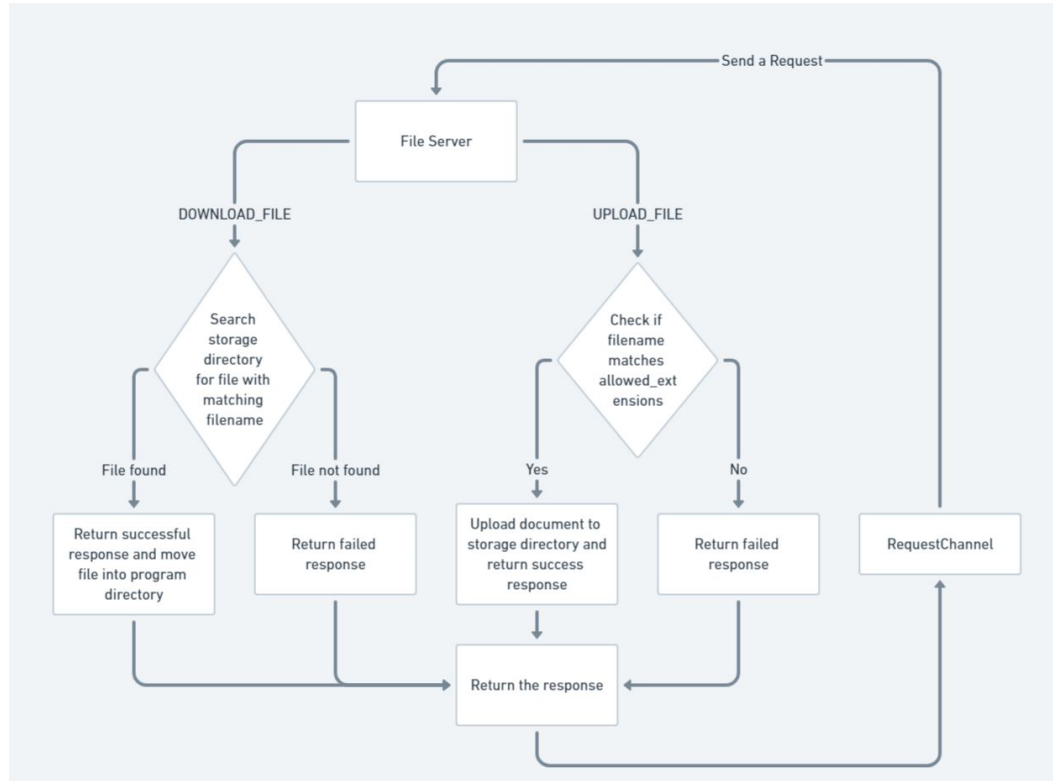
Finance Server



File Server

- The file server has access to the /storage directory
- Only allows so many extensions. Execute using
 - `./fileserver <extension_1> <extension_2> // eg ./file .h .c .txt`
 - If the file server is started without any arguments, no extensions will be allowed (and no files can be uploaded)
- The server can handle 2 request types (defined in the ***“common”*** class):
 - **UPLOAD_FILE** - if file extension does not match, return false. If matching, add to storage directory
 - Note: the file contents will truncate for large files. This is expected behavior, don't worry about it.
 - **DOWNLOAD_FILE** - attempt to find a file with the matching filename, if not found return false
- If some other request type is sent to it, the server responds with false
- You are not required to change the server program

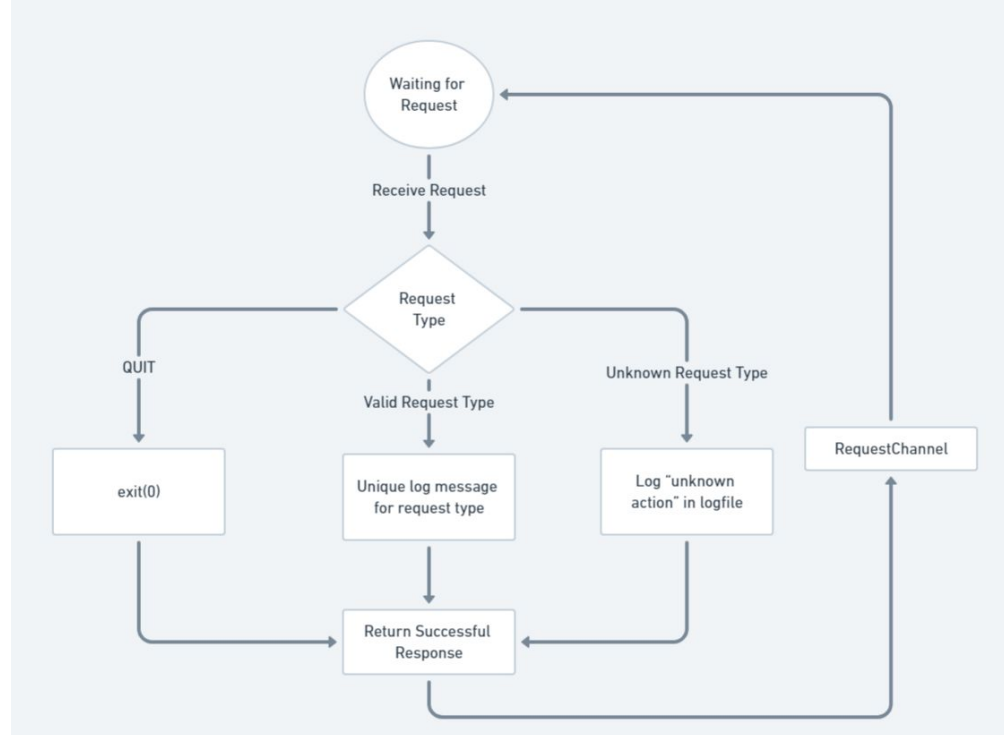
File Server



Logging Server

- The logging server manages a log file
- The log file is passed as an argument when executing.
 - `./logging -f <filename>`
 - The format was built for *.log files (eg example.log), but you can supply any type of file extension
- The server can handle **any** request type (defined in the “**common**” class):
 - All the logging is already implemented for the types of requests in common.h.
 - Some requests use certain fields from the Request in the log message.
 - The output MUST example.log exactly, so supply the relevant fields. example.log in /storage is an example
- QUIT requests are not logged
- You are not required to change the server program

Logging Server



RequestChannel Class (1)

- The RequestChannel class implements an IPC mechanism through which different processes communicate
 - **Only two** connections should be made through a channel. If more are made, there will be undefined behavior.
 - This class is implemented for you and hidden from you and requires no changes
 - The API for this class is exposed through the channel.h file, but the implementation is hidden
- The client and server must connect to the RequestChannel using the **same name** to establish a connection
 - For each name, no more than one connection should be made from each side (one client, one server)
- An example of a successful connection created:
 - The client would join the channel with the common name, “control”, and specify that the client side is joining

```
RequestChannel channel(“example”, RequestChannel::CLIENT_SIDE);
```
 - The server would join the channel with the common name, “control”, and specify that the server side is joining

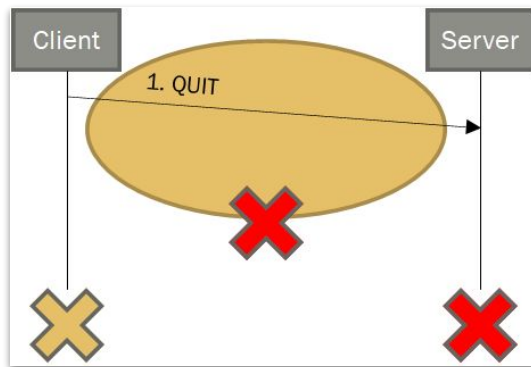
```
RequestChannel channel(“example”, RequestChannel::SERVER_SIDE);
```

RequestChannel Class (2)

- A client or server process that wishes to write to the other will use the `cwrite` function as follows:
 - `Response RequestChannel::send_request (const Request& req);`
 - Only clients should use `send_request`
 - The parameter `req` is a `Response` object
 - This returns a `Response`, which describes the success status and other data
- Servers will communicate using `receive_request()` and `send_response(const Response& resp);`
 - You won't use these functions, so don't worry about them for now

Quit Message

- This is done by setting RequestType in a Request to QUIT and sending it across the channel
- This message closes the channel through which the message was sent
 - For every server created, a QUIT must be sent through the channel to terminate it
- The RequestChannels are destroyed separately. Their destructor is automatically called at the end of the program.
- client.cpp will call wait() at the end of the program
 - The client process will wait until all its children are killed.
 - If any child survived, the parent will continue to wait infinitely



Client

- Lab 2 starter code has the client setup with
 - User interaction
 - Half implemented `print_process_info()`
 - Relevant arguments for starting servers
 - Switch case defining the user's choices and code depending on the response's content
- You must implement the client such that it
 - Runs the servers as child processes (**Note: The autograder won't work without this functionality**)
 - Is successfully able to send requests for each type of action
 - Creates new channels of communication with each server and closes all channels created

Final Remarks

- Create your repo using the link found in Lab 2 doc
- Navigate to the repository and open a terminal
- Run “make” command on one of the terminals
- Run ./client, ./file, ./finance, and ./logging (separately)
 - You should see the client terminate, but the servers will run until you interrupt them due to their need for quit request to exit
- Run “make clean” to remove executable files, and “make distclean” to remove other files

Note on Testing

- For testing locally, a shell script has been provided titled *lab1-tests.sh*
 - The script can be run by running the following `make` command:
`make test`
- The script contains all test cases provided by the autograder and can be modified to include additional tests
 - The intention behind providing the script is to promote good programming/testing practices and to relieve the load on the autograder
- **NOTE: The makefile rules cannot be modified in order for the autograder to run**
 - To enable debugger tools, add the appropriate flags to the `CXXFLAGS` variables in the makefile

fork() + exec()

fork() (1)

- `pid_t fork(); // Function Definition`
- `fork()` creates a new process by duplicating the calling process
 - The new process is termed as the child process
 - The calling process is termed as the parent process
- The child inherits memory from the parent, but has a separate execution context
- `fork()` returns 0 to the child process, and returns the process id (pid) of the child to the parent process

fork() (2)

- Let us consider an example:

```
◦ int main(){  
  
    cout << "There is one process executing this statement" << endl;  
  
    fork();  
  
    cout << "Both the parent and child process will execute this statement" << endl;  
  
}
```

- The output will result in the first cout statement being printed once, and the second cout statement being printed twice
 - Both the parent and child processes execute the same code after fork() is called
 - The order of the child and parent execution is however non-deterministic

fork() (3)

```
int main () {  
    if (fork() == 0)  
        cout << "I am the child" << endl;  
    else  
        cout << "I am the parent" << endl;  
}
```

- In the above example, we can differentiate between the child and parent process
 - We direct the child to execute ***"I am the child"*** as `fork()` returns 0 to child process
 - Similarly, if `fork()` did not return 0, we know it's the parent process in execution

exec()

- `int execvp(const char* file, char const* argv[]);` // Function Definition
 - `const char* file` is the new process we replace the current process with
 - `char const* argv[]` is the array containing the new process' arguments
- The `exec(...)` family of function replaces the current process image with a new process image
- `fork()` + `exec(...)` is most commonly used to get this new process to run as a child of the current process
 - The new process (the child) would now run its own code instructions rather than the instructions inherited from the parent
 - Think how this would relate to the client-server model of communication

Any questions?